

TRẢ LỜI TOÀN BỘ CÂU HỎI VẤN ĐÁP - STAYSAGA

Môn: Các công nghệ mới trong phát triển phần mềm. Tài liệu gồm 57 câu, mỗi câu có 2 hướng trả lời: đáp án chuẩn theo tài liệu/nguồn chính thức và đáp án áp dụng trực tiếp vào app StaySaga.

Nguồn nội bộ dùng để áp dụng StaySaga: README.md, supabase/migrations, docs/qa-full-test-report.md, docs/AI_PROMPTS.md và mã nguồn trong src/.

Câu 1. Sự khác biệt giữa Client Component và Server Component trong Next.js là gì? Khi nào nên dùng loại nào?

1) Đáp án chuẩn: Trả lời chuẩn: Server Component render phía server, gửi HTML đã hoàn chỉnh đến client. Client Component cần tương tác JS phía client (useState, useEffect, event handlers). Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, các trang đọc dữ liệu như /homestays, /bookings, /host và /admin ưu tiên Server Component để đọc Supabase server-side. Các phần có state, click, toast, modal, favorite, wizard và realtime subscription là Client Component.

Câu 2. App Router trong Next.js 13+ khác với Pages Router ở điểm nào? Nêu ít nhất 3 điểm khác biệt chính.

1) Đáp án chuẩn: Trả lời chuẩn: Cấu trúc thư mục (app/ vs pages/), Server Components mặc định, Layouts lồng nhau, Server Actions, Loading/Error files. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng App Router trong src/app với các route /homestays, /bookings, /host, /admin và /api. Dự án tận dụng layout/page theo thư mục, Server Components mặc định, Server Actions trong src/core/*/actions.ts và Route Handler cho endpoint cần URL public.

Câu 3. Server Actions trong Next.js là gì? Cho biết cú pháp khai báo một Server Action.

1) Đáp án chuẩn: Trả lời chuẩn: Async function chạy trên server, có thể gọi trực tiếp từ Client Component. Khai báo với "use server" hoặc trong file action.js/ts. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, các file như src/core/bookings/actions.ts, src/core/auth/actions.ts, src/core/favorites/actions.ts dùng "use server" để xử lý đăng nhập, đặt phòng, hủy/đổi lịch, yêu thích và revalidate dữ liệu sau khi ghi.

Câu 4. Các phương thức Data Fetching nào được hỗ trợ trong Next.js? Khi nào dùng fetch() trực tiếp, khi nào dùng React Query / SWR?

1) Đáp án chuẩn: Trả lời chuẩn: fetch tích hợp sẵn (caching, revalidation), React Query/SWR khi cần client-side fetching, polling, optimistic updates. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga chủ yếu fetch dữ liệu bằng Supabase server client trong Server Component và Server Action. Với phần cần tương tác như bộ lọc, toast, wizard hoặc realtime, dự án dùng Client Component/state; hiện chưa cần React Query vì luồng chính đã nằm ở server và Supabase.

Câu 5. Trong Next.js, file layout.tsx có vai trò gì và khác gì so với page.tsx?

1) Đáp án chuẩn: Trả lời chuẩn: Layout không re-render khi route thay đổi (giữ trạng thái). Dùng cho header, sidebar, providers. Có thể lồng nhau. 1.2. SUPABASE & POSTGRESQL. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: layout.tsx của StaySaga bọc provider, shell và giao diện chung theo nhóm route; page.tsx là nội dung từng màn như danh sách homestay, chi tiết booking hoặc dashboard admin.

Câu 6. Supabase Auth hỗ trợ những phương thức xác thực nào? Liệt kê ít nhất 4 phương thức.

1) Đáp án chuẩn: Trả lời chuẩn: Email/password, Google OAuth, GitHub OAuth, Phone (SMS), Magic Link, SSO. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng Supabase Auth cho đăng ký, đăng nhập, đăng xuất, callback OAuth và quản lý session qua @supabase/ssr. README nêu email/password, Google/Facebook OAuth và quên/đặt lại mật khẩu là các luồng chính.

Câu 7. Row Level Security (RLS) trong Supabase là gì? Mục đích của RLS?

1) Đáp án chuẩn: Trả lời chuẩn: Cơ chế phân quyền ở cấp độ dòng (row) trong database. Đảm bảo user chỉ thấy/sửa được dữ liệu được phép. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga bật RLS cho profiles, homestays, bookings, favorites, reviews và notifications. Policy giúp user chỉ quản lý dữ liệu của mình, host quản lý homestay của mình, admin có quyền quản trị rộng hơn.

Câu 8. Các thành phần chính của Supabase được nhắc đến trong môn học là gì? Liệt kê đầy đủ.

1) Đáp án chuẩn: Trả lời chuẩn: Authentication, Database (PostgreSQL), Storage, Realtime, Edge Functions, Vector. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, Supabase dùng cho Auth, PostgreSQL, RLS, Storage bucket homestay-images và Realtime. Các bảng chính gồm profiles, homestays, homestay_images, amenities, bookings, favorites, reviews, notifications.

Câu 9. Cấu trúc một Row Level Security Policy cơ bản trong Supabase gồm những thành phần nào?

1) Đáp án chuẩn: Trả lời chuẩn: USING clause (điều kiện lọc), WITH CHECK clause (điều kiện insert/update), loại policy (SELECT, INSERT, UPDATE, DELETE), role (authenticated, anon). Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Policy StaySaga dùng auth.uid() để so khớp user_id hoặc owner_id. Ví dụ bookings cho user đọc booking của mình, host đọc booking thuộc homestay mình, còn admin được mở rộng quyền qua role/helper trong migration.

Câu 10. Supabase Realtime hoạt động dựa trên cơ chế nào? Khi nào nên dùng?

1) Đáp án chuẩn: Trả lời chuẩn: Dựa trên PostgreSQL LISTEN/NOTIFY và/hoặc WebSocket. Dùng cho thay đổi dữ liệu real-time, presence, broadcast. 1.3. DOCKER & CONTAINERIZATION. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng RealtimeSubscription để subscribe thay đổi bookings, reviews, homestays, profiles và notifications. Khi có thay đổi, UI refresh hoặc tăng badge thông báo cho guest/host/admin.

Câu 11. Container khác với Virtual Machine (VM) ở điểm nào? Nêu ít nhất 3 điểm.

1) Đáp án chuẩn: Trả lời chuẩn: VM có kernel riêng, container dùng chung kernel host; VM nặng hơn, khởi động chậm hơn; container nhẹ, khởi động nhanh, chi phí tài nguyên thấp hơn. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga đóng gói bằng container để chạy nhất quán giữa máy dev và VPS. Container nhẹ hơn VM, phù hợp app Next.js vì database chính nằm ngoài ở Supabase cloud.

Câu 12. Dockerfile và docker-compose.yml có vai trò gì? Sự khác nhau giữa hai file này?

1) Đáp án chuẩn: Trả lời chuẩn: Dockerfile mô tả cách build một image. docker-compose.yml định nghĩa và chạy multi-container app (services, networks, volumes). Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Dockerfile StaySaga build Next.js standalone; docker-compose.yml định nghĩa service staysaga-web, map \${APP_PORT:-3000}:3000 và truyền biến môi trường Supabase/Stripe vào container.

Câu 13. Multi-stage build trong Docker là gì? Tại sao nên dùng?

1) Đáp án chuẩn: Trả lời chuẩn: Build trong nhiều stage, chỉ copy artifact cần thiết sang stage cuối → image nhỏ hơn, bảo mật tốt hơn, tách biệt build environment. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Dockerfile của StaySaga có stage deps, builder và runner. Runner chỉ copy public, .next/standalone và .next/static rồi chạy node server.js bằng user nextjs, giúp image nhỏ và sạch hơn.

Câu 14. Các câu lệnh Docker cơ bản nào cần biết? (docker build, docker run, docker compose up/down, docker ps, docker logs)

1) Đáp án chuẩn: Trả lời chuẩn: build tạo image, run khởi chạy container, compose up/down quản lý multi-container, ps liệt kê container đang chạy, logs xem output. 1.4. DEPLOYMENT & DEVOPS. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Khi demo/deploy StaySaga thường dùng npm run build, docker compose up --build, docker compose ps, docker compose logs -f staysaga và docker compose down. Khi lỗi production, logs là nơi kiểm tra đầu tiên.

Câu 15. HTTPS và HTTP khác nhau ở điểm nào? Tại sao cần HTTPS cho production?

1) Đáp án chuẩn: Trả lời chuẩn: HTTPS mã hoá dữ liệu bằng TLS/SSL, HTTP truyền plain text. HTTPS bảo mật dữ liệu, xác thực server, tránh man-in-the-middle. Khi vấn đáp, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga có đăng nhập, cookie session, thông tin đặt phòng và có thể thanh toán, nên production bắt buộc dùng HTTPS để bảo vệ token, cookie và dữ liệu khách.

Câu 16. Reverse proxy (ví dụ Nginx) là gì? Trong mô hình deploy với Cloudflare + VPS, reverse proxy đóng vai trò gì?

1) Đáp án chuẩn: Trả lời chuẩn: Server trung gian tiếp nhận request, chuyển tiếp đến app server. Dùng để load balancing, SSL termination, cache, bảo mật. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong mô hình StaySaga, Cloudflare trở domain về VPS, Nginx nhận request 80/443 rồi proxy vào container Next.js port 3000, đồng thời set header X-Forwarded-* và xử lý SSL/redirect HTTPS.

Câu 17. Domain và DNS hoạt động như thế nào? Khi người dùng nhập example.com, quy trình DNS resolution ra sao?

1) Đáp án chuẩn: Trả lời chuẩn: Browser → DNS resolver → Root server → TLD server (.com) → Authoritative nameserver → IP address → Server. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Với StaySaga, domain được cấu hình DNS trên Cloudflare trở tới IP VPS. Người dùng nhập domain, DNS resolve ra Cloudflare/origin, request đi qua Nginx rồi vào container app.

Câu 18. Cloudflare trong mô hình deploy đóng vai trò gì? Nêu ít nhất 3 chức năng.

1) Đáp án chuẩn: Trả lời chuẩn: CDN (tăng tốc), DDoS protection, DNS management, SSL/TLS proxy (Universal SSL), caching, free tier. 1.5. TYPESCRIPT & KIẾN TRÚC WEB. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Cloudflare giúp StaySaga quản lý DNS, bật SSL miễn phí, cache asset tĩnh, chống DDoS cơ bản và ẩn IP VPS, phù hợp mô hình triển khai đồ án.

Câu 19. Sự khác biệt giữa interface và type trong TypeScript là gì? Khi nào dùng interface, khi nào dùng type?

1) Đáp án chuẩn: Trả lời chuẩn: Interface có thể merge (declaration merging), extend được; type linh hoạt hơn (union, intersection). Dùng interface cho object shape, type cho union/intersection/complex types. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng TypeScript cho model/props như booking, homestay, profile. Interface hợp mô tả object row/props; type hợp union role/status như USER | PARTNER | ADMIN hoặc BookingStatus.

Câu 20. SSR, CSR, SSG, ISR là gì? Nêu tên đầy đủ và bản chất mỗi khái niệm.

1) Đáp án chuẩn: Trả lời chuẩn: SSR = Server-Side Rendering, CSR = Client-Side Rendering, SSG = Static Site Generation, ISR = Incremental Static Regeneration. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng server rendering cho các trang cần session và dữ liệu mới như /bookings, /host, /admin. Các phần tương tác như filter, modal, favorite mới chạy CSR ở client.

Câu 21. type annotation và type inference trong TypeScript khác nhau như thế nào? Ví dụ minh họa.

1) Đáp án chuẩn: Trả lời chuẩn: Annotation: khai báo rõ kiểu (let name: string). Inference: TypeScript tự suy kiểu (let name = "A" → string). 1.6. AI TOOLS. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, nên annotation ở boundary như props, payload Server Action và kiểu dữ liệu Supabase; biến cục bộ như nights hoặc totalPrice có thể để TypeScript tự suy luận.

Câu 22. GitHub Copilot và Gemini CLI hoạt động dựa trên nguyên lý nào? Nêu cách sử dụng cơ bản của từng công cụ.

1) Đáp án chuẩn: Trả lời chuẩn: Copilot dùng code context + LLM để suggest code inline. Gemini CLI cho phép tương tác LLM qua terminal, chat, run code. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga có docs/AI_PROMPTS.md ghi các prompt AI dùng để sửa hydration, UI homestays và mock bookings. Có thể nói AI hỗ trợ sinh gợi ý, còn nhóm kiểm tra lại bằng build, lint và test.

Câu 23. Các loại minh chứng sử dụng AI tool mà bạn có thể cung cấp cho giảng viên là gì?

1) Đáp án chuẩn: Trả lời chuẩn: Comment trong code (prompt + response), README (mô tả cách dùng AI), chat history/screenshot, commit message ghi rõ AI hỗ trợ. NHÓM 2 — THÔNG HIỂU Mục tiêu: Sinh viên giải thích, phân tích sự khác biệt, so sánh, diễn giải nguyên lý hoạt động. 2.1. NEXT.JS & KIẾN TRÚC. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Minh chứng AI của StaySaga gồm docs/AI_PROMPTS.md, README, commit history nếu có, tài liệu QA và script sinh báo cáo. Khi vấn đề có thể mở các file này để chứng minh AI được dùng có kiểm soát.

Câu 24. Tại sao Next.js mặc định render bằng Server Component? Lợi ích chính của việc này là gì?

1) Đáp án chuẩn: Trả lời chuẩn: Giảm JavaScript gửi về client (bundle size nhỏ hơn), tốt cho SEO, truy cập trực tiếp server resources (database, file system), cải thiện First Contentful Paint. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga để Server Component mặc định giúp giảm JS client, đọc Supabase bằng cookie server và không lộ logic server. Chỉ các nút favorite, realtime, form động mới chuyển sang client.

Câu 25. Trong Next.js, khi nào bạn cần dùng "use client" directive? Nếu không có directive này thì mặc định component hoạt động ra sao?

1) Đáp án chuẩn: Trả lời chuẩn: Khi cần dùng React hooks (useState, useEffect), event handlers, browser APIs. Mặc định là Server Component (không có "use client"). Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, "use client" dùng cho component có useState/useEffect, router.push, toast, modal, realtime subscription hoặc click handler. Nếu quên directive, Next.js sẽ báo lỗi vì mặc định là Server Component.

Câu 26. Giải thích luồng xử lý một request trong Next.js App Router, từ khi user truy cập URL đến khi nhận được nội dung.

1) Đáp án chuẩn: Trả lời chuẩn: Request → Routing (route matching) → Layout → Loading → Server Component renders → Data fetching → RSC payload gửi về → Client Component hydration → Hiển thị. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Luồng StaySaga: user vào route, App Router match page/layout, Server Component tạo Supabase server client đọc session/cookie, query DB theo RLS, render HTML/RSC rồi hydrate các Client Component như favorite/realtime.

Câu 27. So sánh Server Actions và API Routes trong Next.js. Khi nào dùng cái nào?

1) Đáp án chuẩn: Trả lời chuẩn: Server Actions: gọi trực tiếp như gọi function, đơn giản, type-safe. API Routes: RESTful endpoints, dùng khi cần webhook, external call, flexibility. Server Actions là lựa chọn hiện đại hơn. 2.2. SUPABASE & DATABASE. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng Server Actions cho login/logout, booking, favorite, review và admin updates vì là mutation nội bộ. API Route phù hợp cho webhook thanh toán hoặc endpoint bên ngoài như geocode/places.

Câu 28. Giải thích cách Supabase Auth quản lý session. Access token và refresh token hoạt động như thế nào?

1) Đáp án chuẩn: Trả lời chuẩn: Sau khi đăng nhập, Supabase trả về access token (JWT, hết hạn sau ~1 giờ) và refresh token (lưu trong httpOnly cookie). Khi access token hết hạn, tự động refresh. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng @supabase/ssr để createServerClient đọc/ghi cookie session trong Server Component/Action và proxy.ts để cập nhật session, bảo vệ route quản lý.

Câu 29. Khi nào nên dùng Supabase Storage thay vì lưu file dạng base64 trong database? Ưu điểm của Storage là gì?

1) Đáp án chuẩn: Trả lời chuẩn: Storage: quản lý file riêng, streaming, CDN, quyền truy cập RLS, không làm phình database. Base64 trong DB: tăng kích thước DB, chậm truy vấn. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Ảnh homestay của StaySaga lưu trong bucket homestay-images; database chỉ lưu url/storage_path trong homestay_images. Cách này nhẹ DB, dễ phân quyền upload cho host và để phục vụ ảnh công khai/signed URL.

Câu 30. Mô tả quy trình tạo một RLS Policy để user chỉ đọc được dữ liệu của chính mình trong Supabase.

1) Đáp án chuẩn: Trả lời chuẩn: Tạo bảng có column user_id. Tạo policy SELECT: auth.uid() = user_id. Gắn cho role authenticated. Test bằng cách đăng nhập tài khoản khác. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, bookings có user_id và homestay_id. Policy SELECT cho user dùng auth.uid() = user_id; host dùng EXISTS qua homestays.owner_id; WITH CHECK bảo đảm user chỉ tạo booking của mình.

Câu 31. Sự khác biệt giữa Realtime và polling trong việc cập nhật dữ liệu là gì? Ưu/nhược điểm từng cách?

1) Đáp án chuẩn: Trả lời chuẩn: Polling: gửi request định kỳ (lãng phí bandwidth khi không có thay đổi). Realtime: server push khi có thay đổi (hiệu quả hơn, phức tạp hơn, cần WebSocket). 2.3. DOCKER & DEVOPS. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng Realtime cho booking, review và notification để host/admin thấy thay đổi nhanh. Nếu polling, dashboard phải gọi API định kỳ, tốn request hơn và có độ trễ.

Câu 32. Giải thích tại sao file docker-compose.yml thường dùng volume để mount source code thay vì copy vào image.

1) Đáp án chuẩn: Trả lời chuẩn: Volume mount: code thay đổi → container thấy ngay (hot reload), không cần rebuild image. Copy vào image: mỗi thay đổi cần rebuild, tốn thời gian. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Khi dev StaySaga có thể mount source để hot reload. Docker production hiện copy standalone build vào image để container ổn định, đúng với mục tiêu deploy VPS.

Câu 33. Khi chạy docker compose up, Docker thực hiện những bước nào? Trình bày luồng xử lý.

1) Đáp án chuẩn: Trả lời chuẩn: Đọc docker-compose.yml → Build image (nếu chưa có hoặc có thay đổi) → Pull image (nếu cần) → Tạo network → Tạo và chạy containers → Mount volumes → Cấu hình environment variables. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: docker compose up --build của StaySaga đọc docker-compose.yml, build Dockerfile, tạo service staysaga-web, map port 3000, inject env Supabase/Stripe rồi chạy server.js từ Next standalone.

Câu 34. Tại sao Multi-stage build giúp giảm kích thước Docker image? Giải thích cơ chế.

1) Đáp án chuẩn: Trả lời chuẩn: Stage đầu chứa build tools (Node, compiler). Stage cuối chỉ chứa runtime (node:alpine) và copy artifact từ stage đầu. Loại bỏ dependencies không cần thiết (compiler, dev dependencies). 2.4. DEPLOYMENT & SSL. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Image StaySaga nhỏ vì stage runner chỉ chứa Node runtime, .next/standalone, .next/static và public; npm cache, source thừa và dev dependencies không đi vào image cuối.

Câu 35. Trong mô hình Cloudflare + VPS + Nginx, trình bày luồng request từ browser đến ứng dụng Next.js.

1) Đáp án chuẩn: Trả lời chuẩn: Browser → DNS (Cloudflare) → Cloudflare edge (SSL termination) → VPS IP → Nginx (reverse proxy, port 80/443) → Next.js app (port 3000). Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Luồng production StaySaga: browser -> DNS/Cloudflare -> VPS -> Nginx -> container staysaga-web port 3000 -> Next.js App Router -> Supabase cloud nếu cần dữ liệu -> response quay lại browser.

Câu 36. SSL Certificate (chứng chỉ SSL) hoạt động như thế nào? Giải thích cơ chế mã hoá trong HTTPS handshake.

1) Đáp án chuẩn: Trả lời chuẩn: Client gửi ClientHello → Server gửi certificate (public key + CA signature) → Client xác thực CA → Client tạo pre-master secret, mã hoá bằng public key → Server giải mã → Cả hai tạo shared session key → Mã hoá dữ liệu. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Với StaySaga, TLS bảo vệ cookie Supabase Auth và thông tin booking/checkout. Sau handshake, dữ liệu login, profile và đặt phòng được mã hóa khi truyền qua mạng.

Câu 37. Cloudflare Universal SSL hoạt động như thế nào? Ưu điểm so với tự cài SSL trên VPS?

1) Đáp án chuẩn: Trả lời chuẩn: Cloudflare cấp và renew SSL miễn phí, SSL termination tại Cloudflare edge. Ưu điểm: miễn phí, tự động, bảo mật thêm với Cloudflare features. Nhược điểm: traffic đi qua Cloudflare. 2.5. TYPESCRIPT & KIẾN TRÚC. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga có thể dùng Cloudflare Universal SSL ở edge và Origin Certificate hoặc Let's Encrypt trên VPS. Nên dùng chế độ Full Strict để mã hóa cả đoạn Cloudflare tới VPS.

Câu 38. Giải thích khái niệm generics trong TypeScript. Cho ví dụ một hàm generic đơn giản.

1) Đáp án chuẩn: Trả lời chuẩn: Generic cho phép viết code hoạt động với nhiều kiểu dữ liệu mà vẫn type-safe. Ví dụ: function identity<T>(arg: T): T { return arg; }. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga có thể dùng generic cho helper như Result<T> = { data?: T; error?: string } để các action booking, review, profile có kiểu trả về nhất quán và type-safe.

Câu 39. So sánh SSR và SSG. Khi nào nên dùng SSG, khi nào nên dùng SSR cho một trang web?

1) Đáp án chuẩn: Trả lời chuẩn: SSG: build time render, nhanh, tốt cho nội dung tĩnh (blog, docs), không cần real-time data. SSR: request time render, tốt cho nội dung động (dashboard, profile user). Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trang public ít đổi của StaySaga có thể SSG/ISR; dashboard /host, /admin và /bookings nên SSR vì phụ thuộc session, role và dữ liệu booking mới nhất.

Câu 40. Tại sao nên dùng TypeScript thay vì JavaScript thuần trong dự án Next.js? Liệt kê các lợi ích chính.

1) Đáp án chuẩn: Trả lời chuẩn: Type safety (phát hiện lỗi sớm), autocompletion tốt hơn, refactoring an toàn, documentation tự động qua types, giảm bug runtime. **NHÓM 3 — VẬN DỤNG** Mục tiêu: Sinh viên thiết kế, đánh giá, so sánh lựa chọn, giải quyết vấn đề thực tế trong dự án. 3.1. THIẾT KẾ & KIẾN TRÚC. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng TypeScript để kiểm soát props, role/status, payload booking và dữ liệu Supabase. Điều này giảm lỗi khi refactor route/action và giúp IDE cảnh báo trước khi build.

Câu 41. Trong dự án của bạn, bạn đã thiết kế database schema như thế nào? Vì sao bạn chọn cấu trúc đó thay vì cách khác?

1) Đáp án chuẩn: Trả lời chuẩn: Mô tả các bảng, mối quan hệ (one-to-many, many-to-many), indexes, RLS. Giải thích trade-off: denormalization vs normalization, embedded vs relational. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Schema StaySaga tách profiles, homestays, homestay_images, amenities, homestay_amenities, bookings, favorites, reviews và notifications. Tách bảng giúp query rõ, tránh lặp dữ liệu, đặt khóa ngoại/RLS và mở rộng host/admin dễ hơn.

Câu 42. Khi Supabase không hỗ trợ một tính năng nhất định (ví dụ: complex job queue), bạn sẽ xử lý như thế nào?

1) Đáp án chuẩn: Trả lời chuẩn: Sử dụng Supabase Edge Functions, kết hợp third-party services (Inngest, Trigger.dev), hoặc implement đơn giản với database polling. Giải thích cách chọn và trade-off. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Nếu StaySaga cần job queue như gửi email nhắc check-in, có thể dùng Supabase Edge Functions + cron, Trigger.dev/Inngest hoặc bảng jobs/notifications với worker định kỳ cho quy mô nhỏ.

Câu 43. Bạn đã sử dụng Server Components và Client Components như thế nào trong dự án? Có gặp khó khăn gì khi phân chia không?

1) Đáp án chuẩn: Trả lời chuẩn: Server cho data fetching, static content. Client cho forms, interactive UI. Khó khăn: props serialization, "use client" boundary, data fetching patterns. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga dùng Server Component cho page đọc Supabase và auth gate; Client Component cho FavoriteButton, wizard host, modal đổi giá/ngày, toast và RealtimeSubscription. Khó khăn là ranh giới server-client và props phải serializable.

Câu 44. Thiết kế chiến lược phân quyền (authorization) cho ứng dụng của bạn. Ai được làm gì, kiểm tra ở đâu (client? server? database?)?

1) Đáp án chuẩn: Trả lời chuẩn: RLS ở database (phòng thủ cuối), Server Actions kiểm tra quyền trước khi xử lý, Client-side UI hiển thị/hide theo vai trò. Giải thích tại sao cần kiểm tra ở nhiều lớp. 3.2. DOCKER & DEPLOYMENT. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga phân quyền Guest/USER/PARTNER/ADMIN. Guest xem public; USER đặt phòng, favorite, review; PARTNER quản lý homestay/booking của mình; ADMIN quản lý hệ thống. Quyền được kiểm tra ở UI, server và RLS.

Câu 45. Nếu ứng dụng Next.js của bạn bị crash khi chạy trong Docker container trên VPS, bạn sẽ debug bằng cách nào? Trình bày các bước.

1) Đáp án chuẩn: Trả lời chuẩn: Kiểm tra logs (docker logs, docker compose logs), kiểm tra environment variables, kiểm tra port binding, kiểm tra network (ping, curl), kiểm tra resource (CPU, RAM, disk). Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Nếu StaySaga crash trong Docker, kiểm tra docker compose logs -f staysaga, env Supabase, NEXT_PUBLIC_SITE_URL, APP_PORT, output .next/standalone, npm run build và thử curl localhost:3000 trên VPS.

Câu 46. Khi deploy lên VPS, bạn cần cấu hình Nginx như thế nào để proxy request đến Next.js app? Cho ví dụ cấu hình cơ bản.

1) Đáp án chuẩn: Trả lời chuẩn: Cấu hình location / proxy_pass http://localhost:3000, set headers (Host, X-Real-IP, X-Forwarded-), cấu hình SSL, cấu hình static file serving / next.js image optimization.*. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Nginx cho StaySaga proxy / về http://127.0.0.1:3000 hoặc tên service Docker, set Host/X-Forwarded-Proto/X-Real-IP, redirect HTTP sang HTTPS và tăng client_max_body_size nếu upload ảnh lớn.

Câu 47. Giả sử bạn cần deploy 2 ứng dụng (frontend Next.js + backend NestJS) trên cùng một VPS. Thiết kế cấu hình Docker Compose và Nginx.

1) Đáp án chuẩn: Trả lời chuẩn: 2 services trong docker-compose. Nginx làm reverse proxy: route theo path (/api/ → NestJS, / → Next.js). Cấu hình network, healthcheck. 3.3. SUPABASE & DATA. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Nếu thêm NestJS, Compose có service web:3000 và api:3001 cùng network. Nginx route /api/ tới api:3001, còn / tới web:3000; chỉ Nginx expose 80/443 ra public.

Câu 48. Bạn cần hiển thị danh sách bài viết cho user chưa đăng nhập (anonymous). Nêu cách cấu hình RLS Policy cho trường hợp này.

1) Đáp án chuẩn: Trả lời chuẩn: Policy SELECT cho role anon với điều kiện không có điều kiện (auth.uid() IS NULL hoặc không kiểm tra auth). Hoặc dùng public schema access. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga, danh sách homestay public cho guest nên có SELECT policy cho anon/authenticated với điều kiện is_active = true hoặc status = approved. Dữ liệu draft/ẩn của host không trả cho anon.

Câu 49. Nếu bạn cần upload file lớn (video, 50MB) lên Supabase Storage, bạn cần lưu ý những gì về cấu hình và RLS?

1) Đáp án chuẩn: Trả lời chuẩn: Storage bucket có file size limit (mặc định 50MB, có thể tăng). Cấu hình RLS: ai được upload, ai được đọc. Nên dùng signed URL cho file private. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Với ảnh/video lớn cho homestay, StaySaga cần cấu hình bucket homestay-images về size/MIME, đường dẫn owner_id/property_id, policy cho host upload property của mình và signed URL nếu file private.

Câu 50. Thiết kế hệ thống notification real-time (thông báo khi có tin nhắn mới) sử dụng Supabase Realtime. Trình bày flow từ client đến server.

1) Đáp án chuẩn: Trả lời chuẩn: User A insert notification vào bảng → Supabase Realtime gửi event → User B subscribe channel notifications:user_id=B → Nhận event → Cập nhật UI (toast, badge). 3.4. AI TOOLS & QUY TRÌNH. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Notification StaySaga: khi có booking/message/review mới, server insert vào notifications với user_id người nhận; client subscribe notifications:user_id qua Supabase Realtime; nhận INSERT thì tăng badge, hiện toast và refresh.

Câu 51. Trong quá trình phát triển, bạn đã sử dụng AI tool như thế nào? Chọn một tính năng cụ thể và mô tả prompt bạn dùng để tạo code đó.

1) Đáp án chuẩn: Trả lời chuẩn: Mô tả context (ngôn ngữ, framework), prompt cụ thể, kết quả AI trả về, bạn đã chỉnh sửa gì. Ví dụ: "Viết TypeScript interface cho User model" / "Giải thích RLS policy này". Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Ví dụ StaySaga: dùng AI để thiết kế UI /homestays theo phong cách Booking/Agoda hoặc sửa lỗi hydration App Router. Prompt nêu Next.js App Router, TypeScript, Tailwind và yêu cầu giữ đúng Server/Client boundary.

Câu 52. Nêu ít nhất 3 trường hợp mà AI tool không nên dùng hoặc cần cẩn thận khi dùng trong dự án.

1) Đáp án chuẩn: Trả lời chuẩn: Xử lý logic nghiệp vụ phức tạp (AI dễ sai), security-sensitive code (AI có thể suggest insecure patterns), code mà bạn không hiểu (copy-paste không kiểm tra → không bảo vệ được). Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Trong StaySaga cần cẩn thận khi AI viết RLS, service role, thanh toán, migration database và logic tính tiền/chống double booking. Các phần này phải review thủ công và test bằng nhiều role.

Câu 53. Bạn đã quản lý Git commit của mình như thế nào? Tại sao dùng Conventional Commits? Cho ví dụ commit message.

1) Đáp án chuẩn: Trả lời chuẩn: Conventional Commits: feat: add user authentication, fix: resolve RLS policy bug, docs: update README. Lợi ích: changelog tự động, search, code review dễ hơn. 3.5. TỔNG HỢP & ĐÁNH GIÁ. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: StaySaga nên commit nhỏ theo Conventional Commits như feat: add booking checkout, fix: correct booking RLS, docs: add deployment checklist để lịch sử rõ và dễ review.

Câu 54. Đánh giá 3 điểm mạnh và 3 điểm hạn chế của kiến trúc dự án bạn đã chọn. Nếu làm lại, bạn sẽ thay đổi gì?

1) Đáp án chuẩn: Trả lời chuẩn: Mạnh: đơn giản, maintainable, đúng yêu cầu. Hạn chế: có thể là scalability, complex RLS, performance. Đề xuất cải thiện cụ thể. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Điểm mạnh StaySaga: Next.js + Supabase gọn, RLS bảo mật dữ liệu, Docker standalone dễ deploy. Hạn chế: phụ thuộc schema Supabase cloud, role/RLS phức tạp, một số luồng host/admin cần test thêm.

Câu 55. Trong 6 tiêu chí bắt buộc (Next.js, Supabase, Docker, VPS, GitHub, AI), tiêu chí nào bạn thấy khó nhất? Vì sao? Bạn đã vượt qua như thế nào?

1) Đáp án chuẩn: Trả lời chuẩn: Phân tích cá nhân: có thể là Docker (lần đầu), VPS/SSL (không quen), RLS (logic phức tạp). Nêu cách giải quyết cụ thể. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Với StaySaga, khó nhất là Supabase RLS/schema cloud và Docker/VPS vì lỗi có thể nằm giữa code, config và hạ tầng. Nhóm vượt qua bằng đọc log, viết tài liệu QA, kiểm thử route theo role và ghi bug cụ thể.

Câu 56. Nếu bạn phải thêm một tính năng mới (ví dụ: thanh toán PayPal) vào dự án hiện tại, quy trình thực hiện của bạn sẽ như thế nào?

1) Đáp án chuẩn: Trả lời chuẩn: Thiết kế database (bảng payments), cập nhật RLS, tạo API route / Server Action, tích hợp PayPal SDK, cập nhật UI, viết tests, update docs, deploy. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Thêm PayPal vào StaySaga: tạo bảng payments liên kết bookings, Server Action tạo order, API Route webhook PayPal xác thực server-side, cập nhật booking/payment status, RLS theo role, UI checkout và test sandbox.

Câu 57. Hạn chế/thách thức lớn nhất bạn gặp phải trong quá trình phát triển đồ án là gì? Bạn đã giải quyết bằng cách nào?

1) Đáp án chuẩn: Trả lời chuẩn: Mô tả vấn đề cụ thể (không phải "không biết gì"). Giải pháp: tài liệu, cộng đồng, AI tool, thử nghiệm. Bài học rút ra. Khi vấn đề, nên nêu định nghĩa trước, sau đó nói khi nào dùng và kết luận bằng một ví dụ ngắn để giảng viên thấy mình hiểu bản chất.

2) Theo app StaySaga: Thách thức lớn của StaySaga là đồng bộ codebase với database cloud và phân quyền host/admin. Nhóm xử lý bằng migration, RLS policy, test route theo role, tài liệu qa-full-test-report và workaround demo khi enum role chưa đồng bộ.

Nguồn tham khảo chính

- Next.js Docs: Server/Client Components, App Router, Server Actions, data fetching - <https://nextjs.org/docs>
- Supabase Docs: Auth, Row Level Security, Storage access control, Realtime - <https://supabase.com/docs>
- Docker Docs: Docker Compose và multi-stage builds - <https://docs.docker.com>
- Cloudflare Docs: DNS, proxy, Universal SSL - <https://developers.cloudflare.com>
- MDN Web Docs: HTTP, HTTPS/TLS - <https://developer.mozilla.org>
- TypeScript Handbook: Everyday Types, Generics - <https://www.typescriptlang.org/docs>
- GitHub Docs: GitHub Copilot - <https://docs.github.com/copilot>
- Gemini CLI Docs - <https://google-gemini.github.io/gemini-cli/docs/>